LAB Triển khai và vận hành ứng dụng lên Vercel

> **Đối tượng:** Sinh viên DevOps tập trung vào deploy, config, debug
> **Không yêu cầu:** Hiểu sâu code React hay Express chỉ cần đọc hiểu cơ bản

---

1. Mục tiêu học tập

Sau lab này, người học phải thực hành và chứng minh được:

- Đọc **System Contract** để biết app cần gì mà không cần đọc code
- Cấu hình biến môi trường đúng cho từng môi trường (local / CI / production)
- Deploy monorepo lên Vercel theo đúng thứ tự phụ thuộc
- Debug theo **layer thinking**: xác định layer lỗi trước khi tìm nguyên nhân
- Xử lý **production incident** theo quy trình: hiện tượng layer nguyên nhân fix

---

2. System Contract Đọc trước khi làm bất cứ điều gì

> **DevOps không cần đọc code. DevOps đọc contract.**
> Mọi thứ cần biết để deploy và vận hành app nằm trong section này.

2.1 Kiến trúc và luồng dữ liệu

```
[Trình duyệt]

      HTTP/HTTPS  [Frontend  Vercel Static]

                                            API calls  [Backend  Vercel
Serverless]

                                                                    Supabase
PostgreSQL
                                                                    Supabase
Storage

      WebSocket  [Supabase Realtime]
                                    (frontend subscribe trực tiếp)
```

4 layer cần nắm:

Layer	Thành phần	Vai trò	Fail ảnh hưởng gì
L1 Infra	Vercel	Deploy, routing, env vars	Toàn bộ app không chạy
L2 Backend	Node.js / Express	API, business logic	Frontend không đọc/ghi được dữ liệu
L3 External	Supabase	DB, Storage, Realtime	Mất dữ liệu, upload fail, realtime fail

Layer	Thành phần	Vai trò	Fail ảnh hưởng gì
L4 Frontend	Vite / React	UI, env vars, proxy	Người dùng thấy lỗi / trang trắng

2.2 Required ENV Backend

Khai báo trên Vercel **backend project** và trong file `.env` local:

Biến	Bắt buộc	Mô tả	Lấy ở đâu
`SUPABASE_URL`		URL Supabase project	Supabase Settings API Project URL
`SUPABASE_ANON_KEY`		Public key	Supabase Settings API anon public
`SUPABASE_SERVICE_ROLE_KEY`		Secret key toàn quyền DB	Supabase Settings API service_role
`STORAGE_BUCKET`		Tên bucket lưu file	Tự đặt: `task-attachments`
`FRONTEND_URL`		URL frontend dùng cho CORS	URL Vercel frontend sau khi deploy

2.3 Required ENV Frontend

Khai báo trên Vercel **frontend project** và trong file `.env` local:

Biến	Bắt buộc	Mô tả	Lấy ở đâu
`VITE_SUPABASE_URL`		URL Supabase project	Giống `SUPABASE_URL` phải khai báo lại
`VITE_SUPABASE_ANON_KEY`		Public key	Giống `SUPABASE_ANON_KEY` phải khai báo lại
`VITE_API_BASE_URL`	production	URL backend Vercel	URL backend sau khi deploy để trống khi local

> **Quy tắc Vite:** Chỉ các biến có tiền tố `VITE_` mới được đưa vào bundle frontend.
> Biến không có `VITE_` = `undefined` ở browser, dù đã khai báo đúng trên Vercel.
> Cú pháp `VITE_SUPABASE_URL=${SUPABASE_URL}` chạy local nhưng **không hoạt động trên Vercel** phải điền giá trị trực tiếp.

2.4 Expected Endpoints Backend

Method	Path	Input	Output khi thành công	Dependency
`GET`	`/api/health`		`{"ok":true}`	Không có
`GET`	`/api/tasks`		JSON array tasks	Supabase DB

Method	Path	Input	Output khi thành công	Dependency
`POST`	`/api/tasks`	`{title, description}`	Task object	Supabase DB
`PATCH`	`/api/tasks/:id/status`	`{status}`	Task object	Supabase DB
`POST`	`/api/tasks/:id/attachment`	multipart file	Task với attachment_url	Supabase DB + Storage
`DELETE`	`/api/tasks/:id/attachment`		Task đã xóa attachment	Supabase DB + Storage

2.5 Required External Services

Service	Mục đích	Cần cấu hình
Supabase PostgreSQL	Lưu task data	Tạo bảng `tasks` (xem SQL ở phần 6)
Supabase Storage	Lưu file đính kèm	Tạo bucket `task-attachments`
Supabase Realtime	Push thay đổi DB browser	Bật Replication cho bảng `tasks`

2.6 Deploy Dependencies (thứ tự bắt buộc)

```
[Supabase] được cấu hình [Backend deploy] URL [Frontend deploy] URL [Backend
update CORS]
```

Không thể đảo thứ tự vì mỗi bước cần output của bước trước.

---

3. Chuẩn bị Đăng ký tài khoản

> Làm bước này trước tất cả. Cần 3 tài khoản tất cả miễn phí.

3.1 GitHub

GitHub lưu code và chạy CI tự động.

Đăng ký:

1. https://github.com **Sign up**
2. Nhập email, password, username xác nhận email chọn gói **Free**

Tạo repository:

1. Bấm **+** góc trên phải **New repository**
2. **Name:** `my-task-app` | **Visibility:** Private hoặc Public
3. Tick **Add a README file Create repository**

> Repository Private: Vercel vẫn đọc được khi đăng nhập bằng GitHub.

3.2 Supabase

Supabase cung cấp PostgreSQL, Storage và Realtime tất cả trong một.

Đăng ký:

1. https://supabase.com **Start your project Continue with GitHub**
2. Xác nhận cấp quyền

Tạo project:

1. **New project** chọn Organization điền:
   - **Name:** `task-app`
   - **Database Password:** đặt mạnh **ghi lại ngay, không lấy lại được**
   - **Region:** Southeast Asia (Singapore)
2. **Create new project** chờ 12 phút

Lấy API Keys: Vào **Project Settings** (icon bánh răng) **API** copy 3 giá trị:

Tên trên Supabase	Biến trong `.env`	Loại
Project URL	`SUPABASE_URL`	Public
anon (public)	`SUPABASE_ANON_KEY`	Public dùng ở frontend
service_role	`SUPABASE_SERVICE_ROLE_KEY`	**Secret chỉ dùng ở backend**

> `service_role` **key bypass toàn bộ RLS đọc/ghi/xóa không giới hạn.**
> Nếu lộ: Supabase Settings API **Reset** ngay lập tức.
> Không commit vào Git. Không đặt vào biến `VITE_*`.

3.3 Vercel

Vercel deploy ứng dụng, tích hợp trực tiếp với GitHub.

Đăng ký:

1. https://vercel.com **Sign Up Continue with GitHub** xác nhận cấp quyền

Kiểm tra kết nối: Dashboard có nút **Add New Project**. Không thấy repository **Account Settings Git Integrations GitHub Configure** để cấp quyền.

> Gói Free giới hạn 3 projects. Xóa project cũ không dùng nếu đã đủ.

---

4. Chuẩn bị môi trường local

```
node --version   # >= v20.x.x   https://nodejs.org/
npm --version    # >= 10.x.x
```

```
git --version    # >= 2.x.x     https://git-scm.com/
```

Cấu hình Git lần đầu:

```
git config --global user.name "Tên của bạn"
git config --global user.email "email@example.com"
```

---

5. Phần 1 Cấu hình Supabase

5.1 Tạo bảng `tasks`

Supabase **SQL Editor New query** dán và bấm **Run**:

```
create table tasks (
  id uuid primary key default gen_random_uuid(),
  title text not null,
  description text null,
  status text not null default 'open'
    check (status in ('open', 'in_progress', 'done')),
  attachment_url text null,
  attachment_name text null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Kiểm tra: **Table Editor** phải thấy bảng `tasks`.

5.2 Tạo Storage bucket

Storage New bucket

- **Name:** `task-attachments` *(phân biệt hoa/thường phải khớp `STORAGE_BUCKET` trong `.env`)*
- Tick **Public bucket Save**

5.3 Bật Realtime

Database Replication bật toggle bảng `tasks` (INSERT + UPDATE).

> Nếu không bật: app vẫn chạy, chỉ mất tính năng tự cập nhật giữa các tab.

---

6. Phần 2 Cấu trúc project và `.env`

6.1 Khởi tạo monorepo

```
git clone https://github.com/username/my-task-app
cd my-task-app
```

Cấu trúc:

```
my-task-app/
 .env                    KHÔNG commit  chứa secrets
 .env.example            COMMIT  mẫu không có giá trị thực
 .gitignore
 .github/workflows/ci.yml
 backend/
      server.js
      package.json
      vercel.json
      src/
            lib/supabase.js
            controllers/taskController.js
 frontend/
       vite.config.ts
       package.json
       vercel.json
       src/
             lib/api.ts
             lib/supabase.ts
             hooks/useRealtimeTasks.ts
```

6.2 `.gitignore`

```
.env
.env.local
.env*.local
node_modules/
**/node_modules/
dist/
build/
```

> Nếu lỡ commit `.env`: xóa file `git rm --cached .env` commit lại. Nhưng secret đã lộ trong lịch sử **phải reset key trên Supabase ngay**.

6.3 `.env` (root, không commit)

```
# Lấy từ: Supabase  Project Settings  API
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=sb_publishable_...
SUPABASE_SERVICE_ROLE_KEY=sb_secret_...
```

```
STORAGE_BUCKET=task-attachments
FRONTEND_URL=http://localhost:5173

# Phải có tiền tố VITE_  Vercel không hỗ trợ ${...} interpolation
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_...
VITE_API_BASE_URL=
```

6.4 `.env.example` (commit được)

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=sb_publishable_your_anon_key
SUPABASE_SERVICE_ROLE_KEY=sb_secret_your_service_role_key

STORAGE_BUCKET=task-attachments
FRONTEND_URL=http://localhost:5173

VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_your_anon_key
VITE_API_BASE_URL=
```

---

7. Phần 3 Xây dựng Backend

7.1 Khởi tạo

```
cd backend
npm init -y
npm install express cors multer @supabase/supabase-js dotenv
```

`package.json` thêm:

```
{
  "type": "module",
  "scripts": {
    "dev": "node --watch server.js",
    "start": "node server.js",
    "lint": "eslint src",
    "test": "vitest run --passWithNoTests"
  },
  "engines": { "node": ">=20" }
}
```

7.2 Kết nối Supabase `src/lib/supabase.js`

```
import { createClient } from '@supabase/supabase-js';
import 'dotenv/config';

export const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY
);
```

7.3 `server.js`

```
import express from 'express';
import cors from 'cors';
import 'dotenv/config';
import taskRouter from './src/controllers/taskController.js';

const app = express();

const allowedOrigins = (process.env.FRONTEND_URL || 'http://localhost:5173')
  .split(',').map(o => o.trim());

app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) return callback(null, true);
    callback(new Error(`CORS blocked: ${origin}`));
  },
  credentials: true,
}));

app.use(express.json());
app.get('/api/health', (_req, res) => res.json({ ok: true }));
app.use('/api', taskRouter);

if (process.env.NODE_ENV !== 'production') {
  app.listen(process.env.PORT || 3001,
    () => console.log(`Backend: http://localhost:${process.env.PORT || 3001}`));
}

export default app;
```

7.4 `vercel.json` cho backend

```
{
  "version": 2,
  "builds": [{ "src": "server.js", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "server.js" }]
}
```

> `"src": "server.js"` phải khớp tên file entry point. Đổi tên file = phải sửa cả đây.

---

8. Phần 4 Xây dựng Frontend

8.1 Khởi tạo

```
cd frontend
npm create vite@latest . -- --template react-ts
npm install @supabase/supabase-js
```

8.2 `vite.config.ts`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  envDir: path.resolve(__dirname, '..'),  // đọc .env từ root monorepo
  plugins: [react()],
  server: {
    proxy: { '/api': 'http://localhost:3001' },  // local dev only
  },
});
```

8.3 `src/lib/supabase.ts`

```
import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);
```

8.4 `src/lib/api.ts`

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? '';

export async function apiFetch<T>(path: string, options?: RequestInit): Promise<T>
{
  const res = await fetch(`${API_BASE_URL}${path}`, options);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

8.5 `vercel.json` cho frontend

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

---

9. Phần 5 Lint và Test

9.1 Backend

```
cd backend
npm install --save-dev eslint @eslint/js globals vitest
```

`backend/eslint.config.js`:

```
import js from '@eslint/js';
import globals from 'globals';

export default [
  js.configs.recommended,
  { languageOptions: { globals: globals.node } },
  {
    files: ['src/__tests__/**'],
    languageOptions: {
      globals: { ...globals.node, vi: true, describe: true, it: true, test: true,
expect: true },
    },
  },
];
```

`backend/vitest.config.js`:

```
import { defineConfig } from 'vitest/config';
export default defineConfig({ test: { globals: true, environment: 'node' } });
```

9.2 Frontend

```
cd frontend
npm install --save-dev vitest @testing-library/react @testing-library/jest-dom
jsdom
```

Thêm vào `vite.config.ts`:

```
test: { globals: true, environment: 'jsdom', setupFiles: ['./src/test/setup.ts'] }
```

`src/test/setup.ts`:

```
import '@testing-library/jest-dom';
```

`tsconfig.json` thêm `"vitest/globals"` vào `compilerOptions.types`.

9.3 Kiểm tra local trước khi push

```
cd backend && npm run lint && npm test
cd frontend && npm run lint && npm run typecheck && npm test && npm run build
```

---

10. Phần 6 GitHub Actions CI

`.github/workflows/ci.yml`:

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: backend/package-lock.json
      - run: npm ci
      - run: npm run lint
      - run: npm test

  frontend:
    runs-on: ubuntu-latest
```

```yaml
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
        env:
          VITE_SUPABASE_URL: ${{ secrets.VITE_SUPABASE_URL }}
          VITE_SUPABASE_ANON_KEY: ${{ secrets.VITE_SUPABASE_ANON_KEY }}
          VITE_API_BASE_URL: ${{ secrets.VITE_API_BASE_URL }}
```

Thêm GitHub Secrets (Repository Settings Secrets Actions):

Name	Value
`VITE_SUPABASE_URL`	URL Supabase
`VITE_SUPABASE_ANON_KEY`	Anon key
`VITE_API_BASE_URL`	URL backend *(điền sau khi deploy)*

> **GitHub Secrets Vercel Environment Variables.** Phải khai báo ở cả 2 nơi riêng biệt.

---

11. Phần 7 Deploy lên Vercel

> File `.env` không tồn tại trên Vercel. Mọi biến phải khai báo thủ công trên dashboard.

11.1 Deploy Backend

1. Vercel **Add New Project** Import repository
2. **Root Directory:** `backend` | **Framework:** Other
3. Thêm ENV vars (theo bảng ở mục 2.2) `FRONTEND_URL` để trống
4. **Deploy** copy URL backend
5. Kiểm tra: `https://[backend-url]/api/health {"ok":true}`

11.2 Deploy Frontend

1. **Add New Project** cùng repository
2. **Root Directory:** `frontend` | **Framework:** Vite
3. Thêm ENV vars (theo bảng ở mục 2.3) `VITE_API_BASE_URL` = URL backend
4. **Deploy** copy URL frontend

11.3 Cập nhật CORS và Redeploy Backend

1. Backend project **Settings Environment Variables**
2. `FRONTEND_URL` = URL frontend
3. **Deployments Redeploy**

11.4 Kiểm tra sau deploy

```
https://[backend]/api/health    {"ok":true}
https://[backend]/api/tasks     [] hoặc JSON array
https://[frontend]/             UI hiển thị, không lỗi console đỏ
```

---

12. Layer Thinking Cách debug theo DevOps

> **Nguyên tắc:** Xác định đúng layer trước, không đoán mò.

12.1 Sơ đồ layer

```
Hiện tượng (triệu chứng)



  L4  Frontend        Lỗi hiển thị? console đỏ? biến undefined?

            Nếu không phải Frontend


  L3  Backend API     API trả sai? HTTP 4xx/5xx? CORS block?

            Nếu API OK


  L2  External        Supabase kết nối được? Storage OK? Realtime bật?

            Nếu External OK


  L1  Infra           Vercel build thành công? ENV vars đúng? Deploy xong chưa?
```

12.2 Quy trình debug 4 bước

Bước 1 Quan sát hiện tượng, không đoán nguyên nhân

- Lỗi hiện ở đâu? (console browser / Vercel logs / network tab)
- HTTP status code là gì? (200, 404, 500, CORS error)
- Xảy ra khi nào? (load trang / bấm nút / sau deploy)

Bước 2 Xác định layer

- Test từng layer một bằng cách gọi trực tiếp
- Backend: curl hoặc mở thẳng URL API trong browser
- Frontend: DevTools Network tab

Bước 3 Khoanh vùng trong layer

- Layer đó có nhiều thành phần thành phần nào fail?
- Backend fail: DB hay Storage hay code logic?
- Frontend fail: env var hay API call hay render?

Bước 4 Tìm nguyên nhân gốc rễ

- Kiểm tra ENV vars của layer đó
- Kiểm tra logs của layer đó
- Đọc error message đừng bỏ qua

12.3 Công cụ debug theo layer

Layer	Công cụ	Cách dùng
Frontend	DevTools Console	Xem lỗi JS, biến undefined
Frontend	DevTools Network	Xem request/response, status code
Backend	Vercel Deployments Functions Logs	Xem runtime error
Backend	Gọi trực tiếp URL API	Kiểm tra độc lập, không qua frontend
External	Supabase Logs	Xem query fail, auth error
Infra	Vercel Deployments	Xem build log, deploy status

---

13. Production Incidents Thực hành vận hành

> **Framing:** Bạn là DevOps. Hệ thống đang chạy production. Người dùng báo lỗi.
> **Nhiệm vụ:** Xác định layer, tìm nguyên nhân, fix và verify.

Incident 1 "API lỗi sau khi tôi update config"

Báo cáo nhận được:

> "Trước đây chạy bình thường. Vừa sửa gì đó trên Vercel xong thì `/api/tasks` báo 500."

Setup: Vào Vercel backend ENV sửa `SUPABASE_URL` thành URL sai Redeploy

Quy trình xử lý:

1. Kiểm tra `/api/health` còn OK không?
2. Kiểm tra `/api/tasks` HTTP status và error message
3. Xác định layer: Backend hay External?

4. Vào Vercel Functions Logs đọc stack trace
5. Tìm nguyên nhân, fix, verify

Kết quả mong đợi sau fix: `/api/tasks` trả JSON array
Bài học: Phân biệt "code lỗi" với "config lỗi" production incident phần lớn là config.

Incident 2 "Frontend deploy xong nhưng trắng trang"

Báo cáo nhận được:

> "Deploy frontend lên Vercel xong, mở URL thì trắng trang. Không thấy gì cả."

Setup: Vercel frontend ENV xóa `VITE_SUPABASE_URL` Redeploy

Quy trình xử lý:

1. Mở URL frontend xác nhận hiện tượng
2. F12 Console đọc lỗi đỏ
3. Lỗi ở layer nào? (Frontend biến undefined)
4. Kiểm tra Vercel frontend ENV vars
5. Fix và Redeploy

Kết quả mong đợi sau fix: UI hiển thị đúng
Bài học: `VITE_` prefix là bắt buộc không có = `undefined` ở browser.

Incident 3 "App hiển thị được nhưng không load task"

Báo cáo nhận được:

> "Trang load được rồi. Nhưng chỗ danh sách task hiện lỗi 'Failed to fetch'. Network tab hiện lỗi JSON parse."

Setup: Vercel frontend xóa `VITE_API_BASE_URL` Redeploy

Quy trình xử lý:

1. F12 Network tab xem request `/api/tasks`
2. Response là gì? (HTML hay JSON?)
3. Xác định: frontend đang gọi URL nào?
4. Kiểm tra `VITE_API_BASE_URL` trên Vercel
5. Fix và verify bằng Network tab

Kết quả mong đợi sau fix: Network tab thấy JSON array
Bài học: Vite proxy `/api localhost:3001` chỉ chạy local. Production không có proxy.

Incident 4 "Frontend gọi API bị block, backend nhận được request"

Báo cáo nhận được:

> "F12 Console hiện: `Access to fetch at 'https://backend.vercel.app/api/tasks' from origin 'https://frontend.vercel.app' has been blocked by CORS policy`. Nhưng team backend bảo server nhận được request rồi."

Setup: Vercel backend `FRONTEND_URL` = URL sai Redeploy

Quy trình xử lý:

1. Đọc CORS error origin là gì, URL backend là gì
2. CORS error xảy ra ở đâu? (browser, không phải server)
3. Kiểm tra backend `FRONTEND_URL` có khớp origin không
4. Fix và Redeploy backend

Kết quả mong đợi sau fix: Không còn CORS error
Bài học: CORS error = browser block server vẫn nhận request. Fix ở backend config, không phải frontend.

Incident 5 "Tạo task OK, upload file fail"

Báo cáo nhận được:

> "Tạo task mới vẫn được. Nhưng khi upload file đính kèm thì lỗi. Các task cũ vẫn load bình thường."

Setup: Vercel backend `STORAGE_BUCKET=wrong-bucket` Redeploy

Quy trình xử lý:

1. Xác định scope: chỉ upload fail, hay toàn bộ?
2. Kiểm tra Vercel Functions Logs khi upload
3. Error message liên quan đến Storage hay DB?
4. Kiểm tra `STORAGE_BUCKET` và tên bucket trên Supabase
5. Fix và test upload

Kết quả mong đợi sau fix: Upload file thành công, URL lưu trong DB
Bài học: Partial failure một dependency fail không kéo sập toàn hệ thống. Khoanh vùng trước khi fix.

Incident 6 "Realtime không còn hoạt động"

Báo cáo nhận được:

> "Trước đây mở 2 tab thì tab 2 tự cập nhật. Hôm nay phải F5 mới thấy. Task vẫn tạo được bình thường."

Setup: Comment out `useRealtimeTasks(...)` trong frontend component

Quy trình xử lý:

1. Xác định: CRUD còn hoạt động không? (có)
2. Reload có thấy dữ liệu đúng không? (có)
3. Chỉ mất gì? (real-time update)
4. Kiểm tra Supabase Realtime có bật không
5. Kiểm tra frontend có subscribe không (DevTools WebSocket connections)

Kết quả mong đợi sau fix: Tab 2 tự cập nhật khi tab 1 tạo task
Bài học: Realtime là lớp "delivery" mất realtime không mất dữ liệu. DB là source of truth.

Incident 7 "CI xanh nhưng Vercel lỗi" *(bonus)*

Báo cáo nhận được:

> "GitHub Actions CI chạy xanh hết. Nhưng Vercel deploy xong thì frontend không load được, lỗi Supabase."

Setup: Thêm secrets vào GitHub Actions, nhưng không thêm vào Vercel frontend ENV vars.

Quy trình xử lý:

1. CI xanh = build đúng với biến mock từ GitHub Secrets
2. Vercel build lại = dùng Vercel ENV vars thiếu biến
3. Kiểm tra Vercel frontend Settings ENV vars
4. Đối chiếu với `.env.example` thiếu gì?
5. Thêm vào Vercel Redeploy

Kết quả mong đợi sau fix: Frontend load đúng
Bài học: CI pass production OK. GitHub Secrets và Vercel ENV vars là 2 hệ thống riêng biệt.

---

14. Bảng xử lý lỗi nhanh

Hiện tượng	Layer	Kiểm tra	Fix
Trang trắng hoàn toàn	L4 Frontend	F12 Console	Xem lỗi JS, kiểm tra `VITE_*` vars
"supabaseKey is required"	L4 Frontend	Vercel frontend ENV	Thêm `VITE_SUPABASE_ANON_KEY`
`Unexpected token '<'` khi load task	L4L3	Network tab, xem response	Thêm `VITE_API_BASE_URL` = URL backend
CORS error trong console	L3 Backend	Vercel backend ENV	Sửa `FRONTEND_URL` = URL frontend
`/api/tasks` trả 500	L3 Backend	Vercel Functions Logs	Kiểm tra Supabase ENV vars
`/api/health` OK, `/api/tasks` fail	L2 External	Supabase Logs	Kiểm tra DB kết nối, bảng tồn tại
Upload fail, CRUD OK	L2 External	Vercel Logs	Kiểm tra `STORAGE_BUCKET`, bucket tồn tại
Realtime không hoạt động	L2 External	Supabase Replication	Bật toggle bảng `tasks`

Hiện tượng	Layer	Kiểm tra	Fix
404 mọi route frontend	L1 Infra	`vercel.json`	Thêm rewrite `/* /index.html`
`${VAR}` hiện literal string	L1 Infra	Vercel ENV value	Dùng giá trị trực tiếp
CI pass, Vercel lỗi	L1 Infra	Vercel ENV vs GitHub Secrets	Khai báo ở cả 2 nơi

---

15. Tiêu chí nghiệm thu

Chức năng:

- ☐ Tạo task, xem danh sách
- ☐ Upload file Supabase Table Editor chỉ thấy URL, không thấy binary
- ☐ 2 tab: tab A tạo task tab B cập nhật không cần F5

CI/CD:

- ☐ `npm run lint && npm test` thành công ở cả 2 project
- ☐ Push GitHub Actions CI xanh
- ☐ Có `.env.example` không chứa giá trị thực

Deploy:

- ☐ `https://[backend]/api/health {"ok":true}`
- ☐ `https://[backend]/api/tasks` JSON array
- ☐ Frontend trên Vercel không lỗi console đỏ

Debug (đánh giá kỹ năng):

- ☐ Làm xong ít nhất 5/7 Incidents trong phần 13
- ☐ Giải thích được: gặp hiện tượng X xác định layer nào tìm ở đâu

---

16. Câu hỏi kiểm tra

1. Nhìn vào System Contract (mục 2), DevOps biết được những gì mà không cần đọc code?
2. `SUPABASE_URL` và `VITE_SUPABASE_URL` phải khai báo riêng giải thích cơ chế Vite đằng sau.
3. Cú pháp `${SUPABASE_URL}` chạy local nhưng fail trên Vercel nguyên nhân ở layer nào?
4. CORS error xảy ra ở layer nào? Server có nhận được request không? Fix ở đâu?
5. CI pass nhưng Vercel fail ví dụ cụ thể và nguyên nhân tại sao có thể xảy ra.
6. Supabase Storage down vẽ sơ đồ: tính năng nào vẫn chạy, tính năng nào fail, layer nào bị ảnh hưởng.
7. Thứ tự deploy: backend frontend update CORS nếu đảo thứ tự thì điều gì xảy ra cụ thể ở từng bước?
8. Bạn nhận được báo lỗi: "Frontend load được nhưng không thấy task". Mô tả 4 bước debug theo layer thinking.
9. Tại sao `SUPABASE_SERVICE_ROLE_KEY` không được dùng ở frontend dù về mặt kỹ thuật hoàn toàn làm được?
10. Có 3 môi trường: production, staging, preview mỗi môi trường frontend cần gọi backend khác nhau. Bạn cấu hình như thế nào trên Vercel?